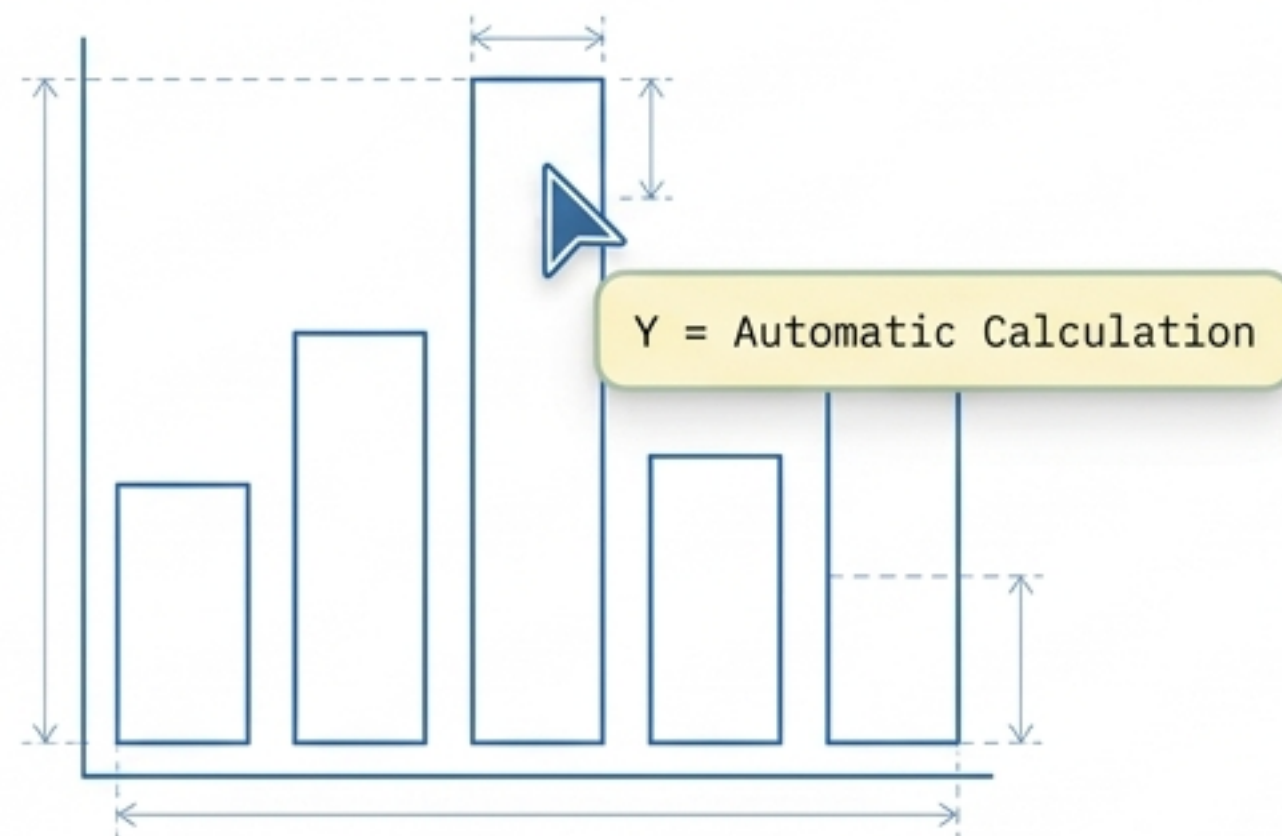


The Seaborn Blueprint: Mastering Categorical Data

A visual guide to Countplots,
Barplots, and Statistical Aggregation.

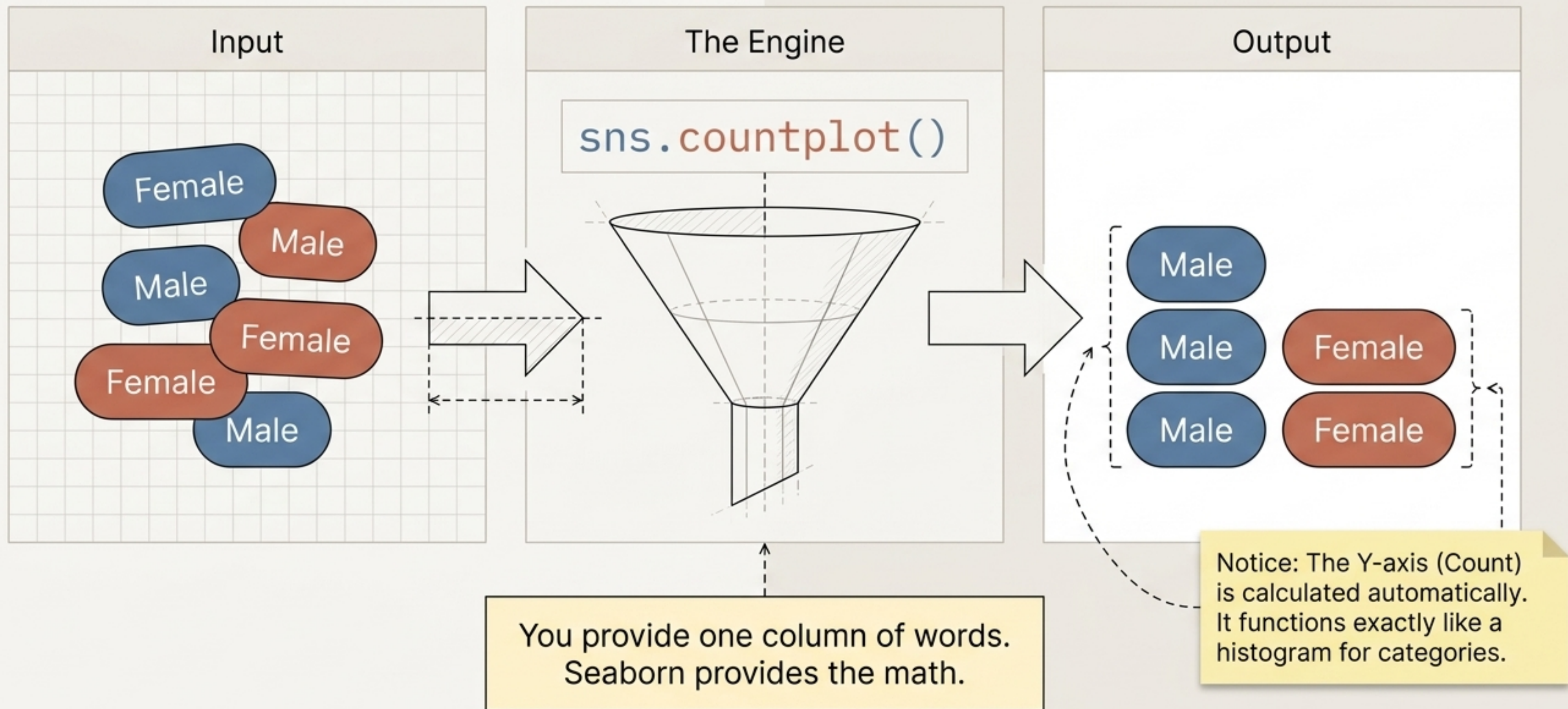


The Seaborn Decision Matrix

	<code>sns.countplot()</code>	<code>sns.barplot()</code>
Primary Question	How many?	What is the average?
Required Inputs	1 Axis (Categorical)	2 Axes (Categorical + Numerical)
Internal Mechanism	Frequency Counting	Statistical Aggregation
Visual Output	Solid Categorical Bars	Aggregated Bars with Error Lines

Data Structure

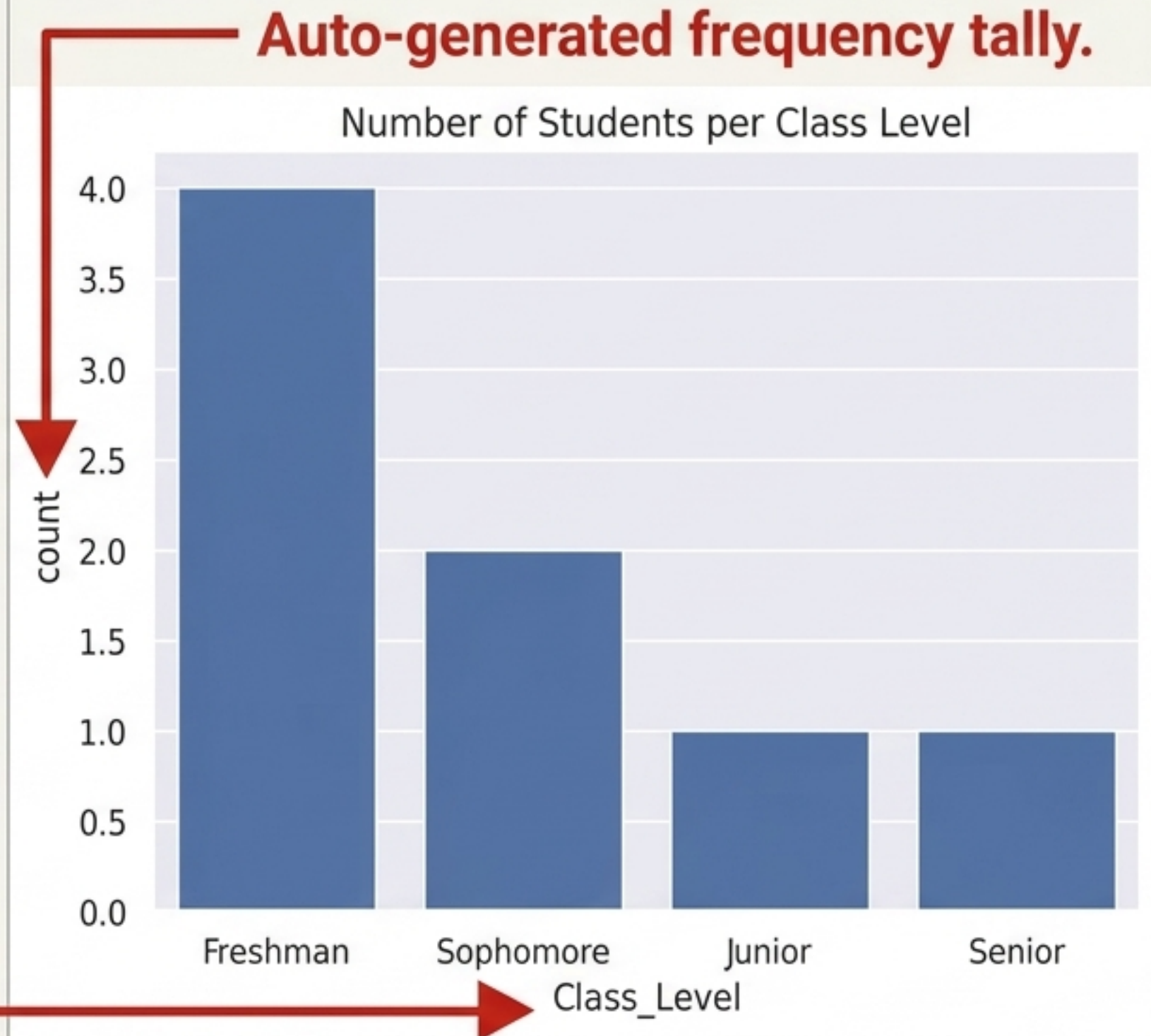
Concept Deep Dive: The Automatic Tally Counter



The Editor

```
data = {  
    'Class_Level': ['Freshman',  
                    'Sophomore', 'Junior', 'Senior']  
}  
df = pd.DataFrame(data)  
  
sns.countplot(data=df, x='Class_Level')
```

The Output Console

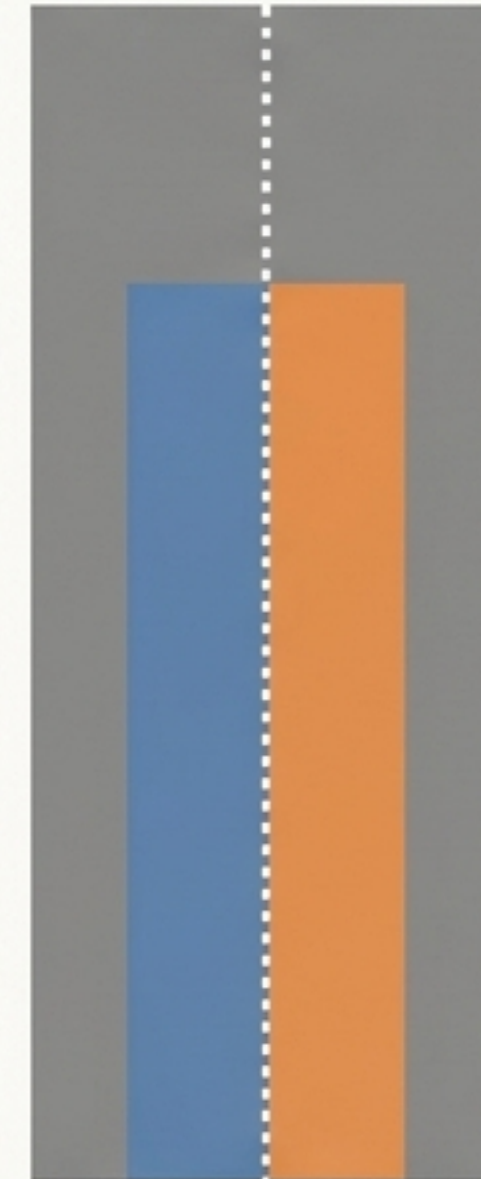


The Editor

```
sns.countplot(data=df, x='Class_Level',  
              + hue='Gender')  
)
```

The Output Console

Hue splits a single bar into comparative sub-categories. Seaborn now calculates the gender breakdown within each class instantly.

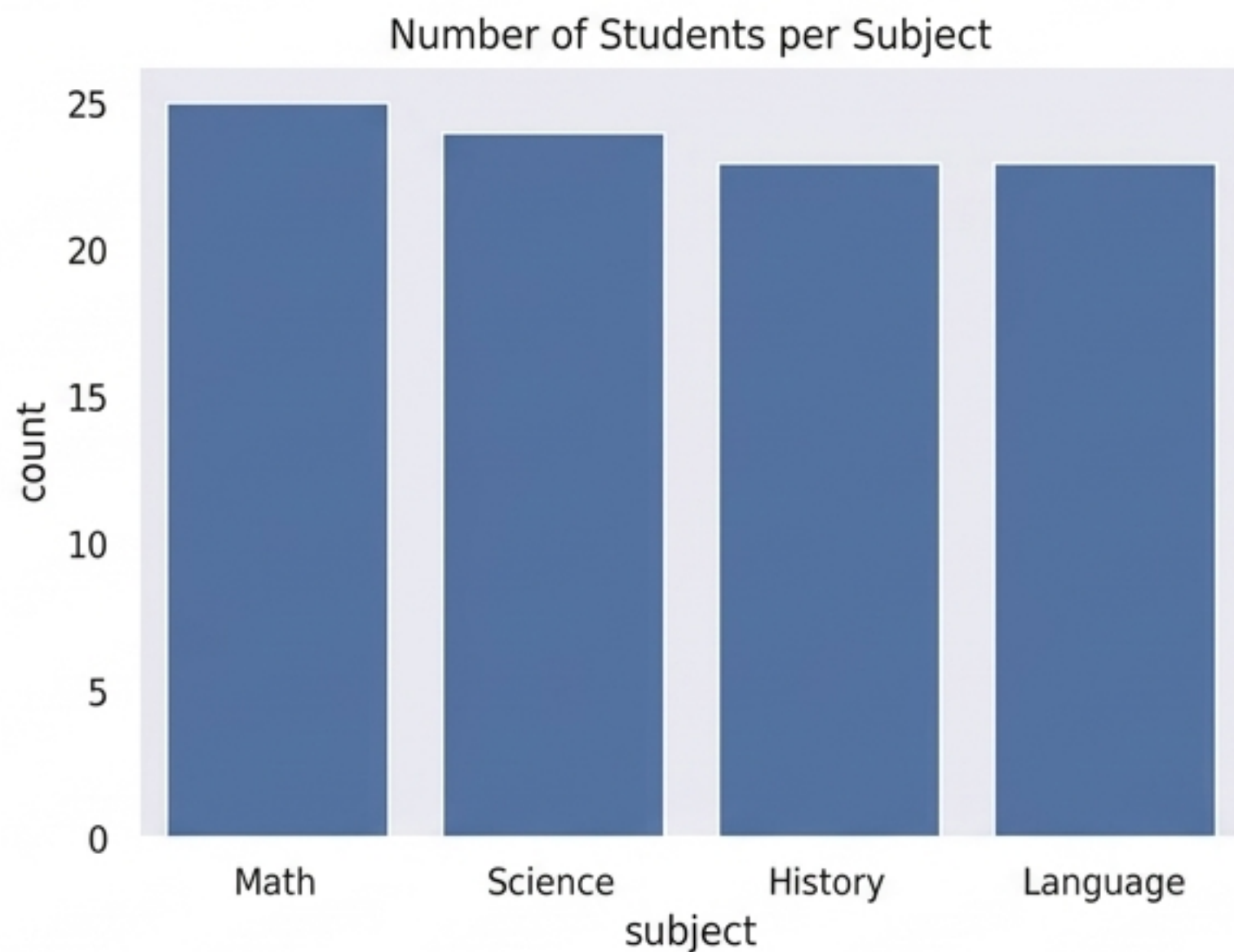


Pro-Tip:

Next steps: Learn to automatically sort bars from tallest to shortest, or write exact numbers on top.

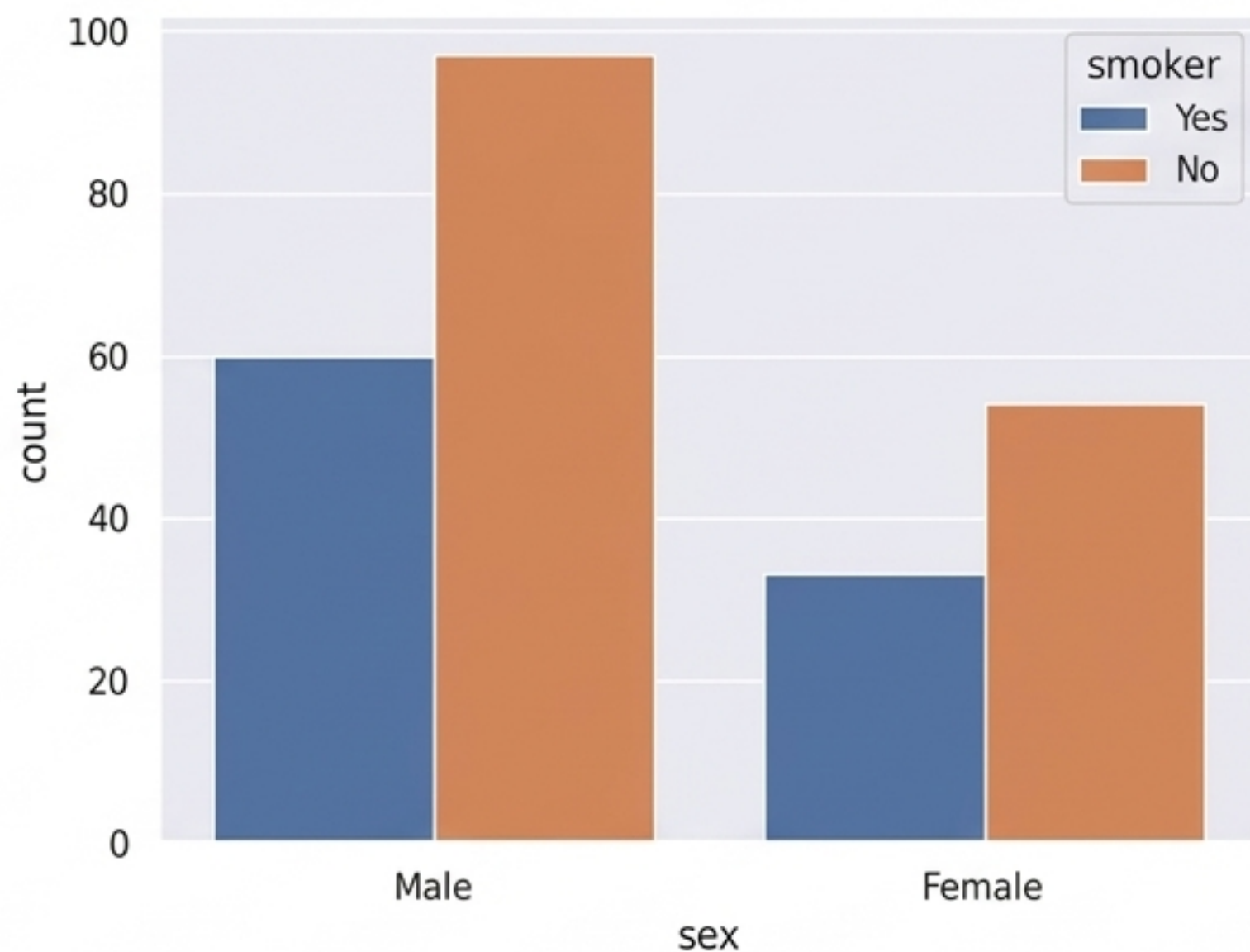
Count Plot Iterations

```
sns.countplot(data=student, x='subject')
```



Basic 1D tally of student subjects.

```
sns.countplot(data=tips_data, x='sex', hue='smoker')
```

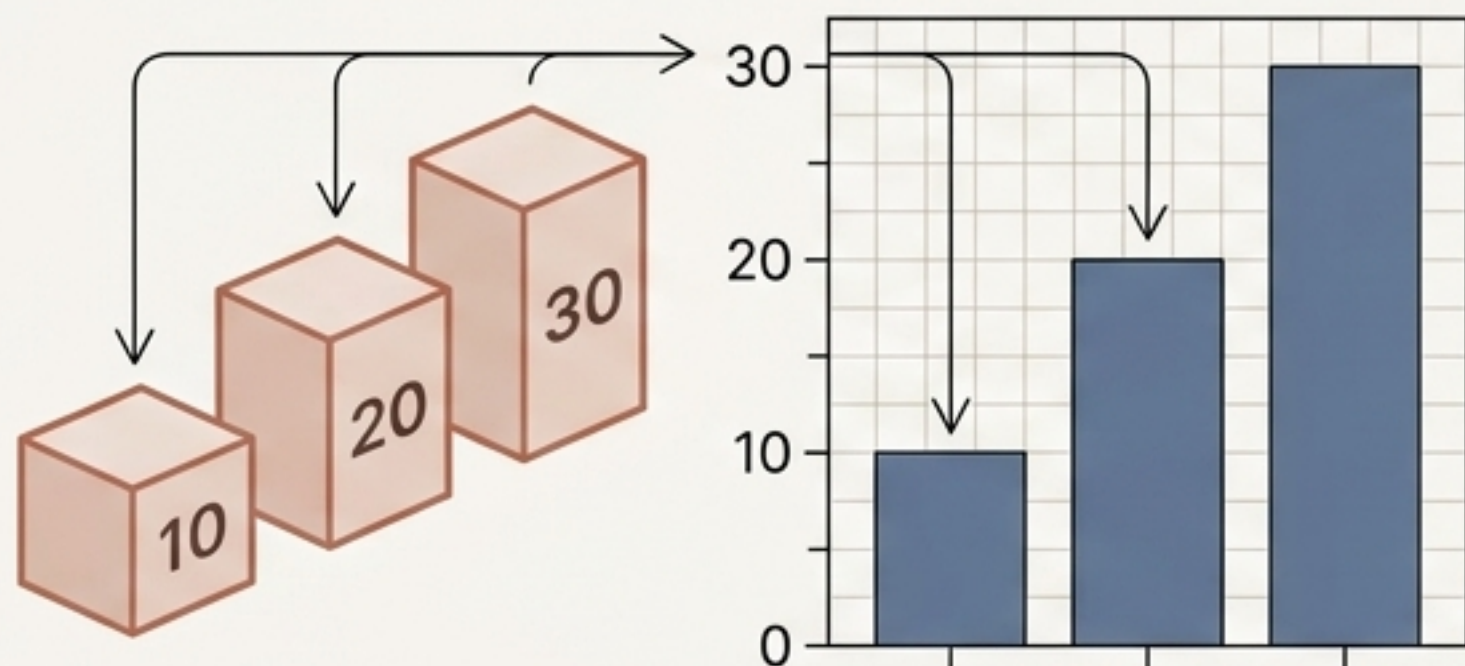


2D comparative tally across restaurant patrons.

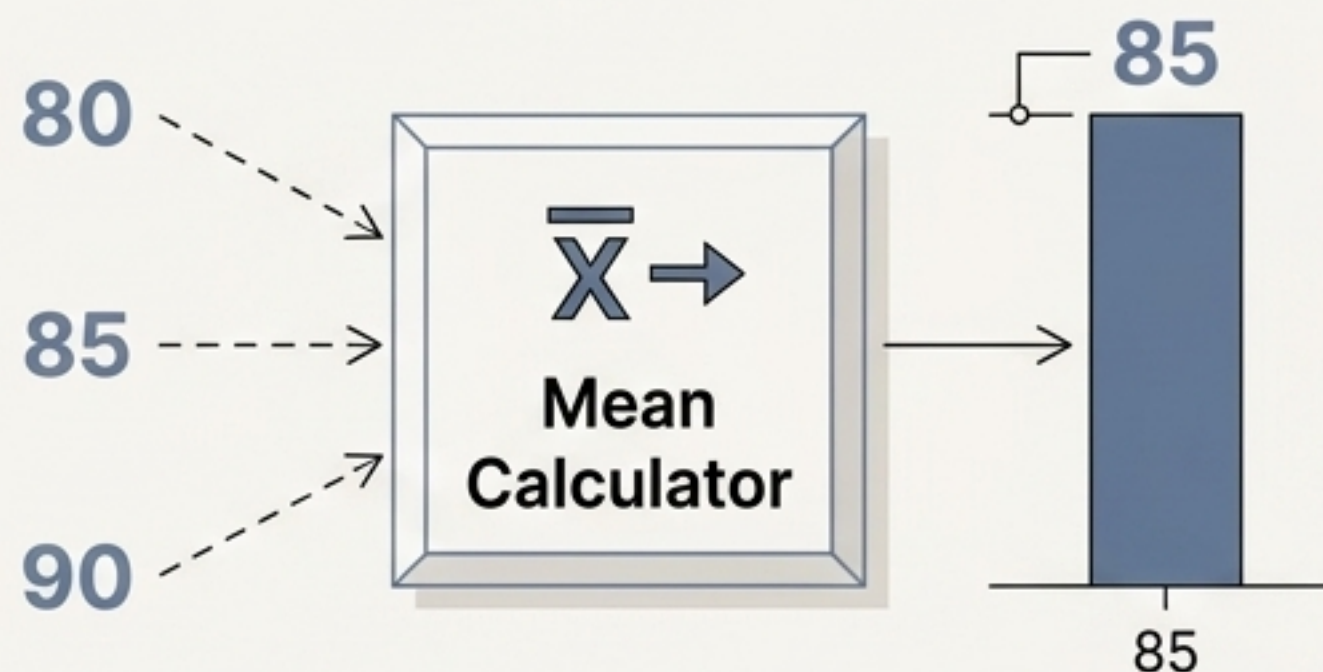
Concept Deep Dive: The Statistical Bar Plot

A standard Matplotlib bar chart plots exact numbers.
Seaborn's **barplot()** is smarter—it acts as a statistical engine.

Literal Plotting (Matplotlib)

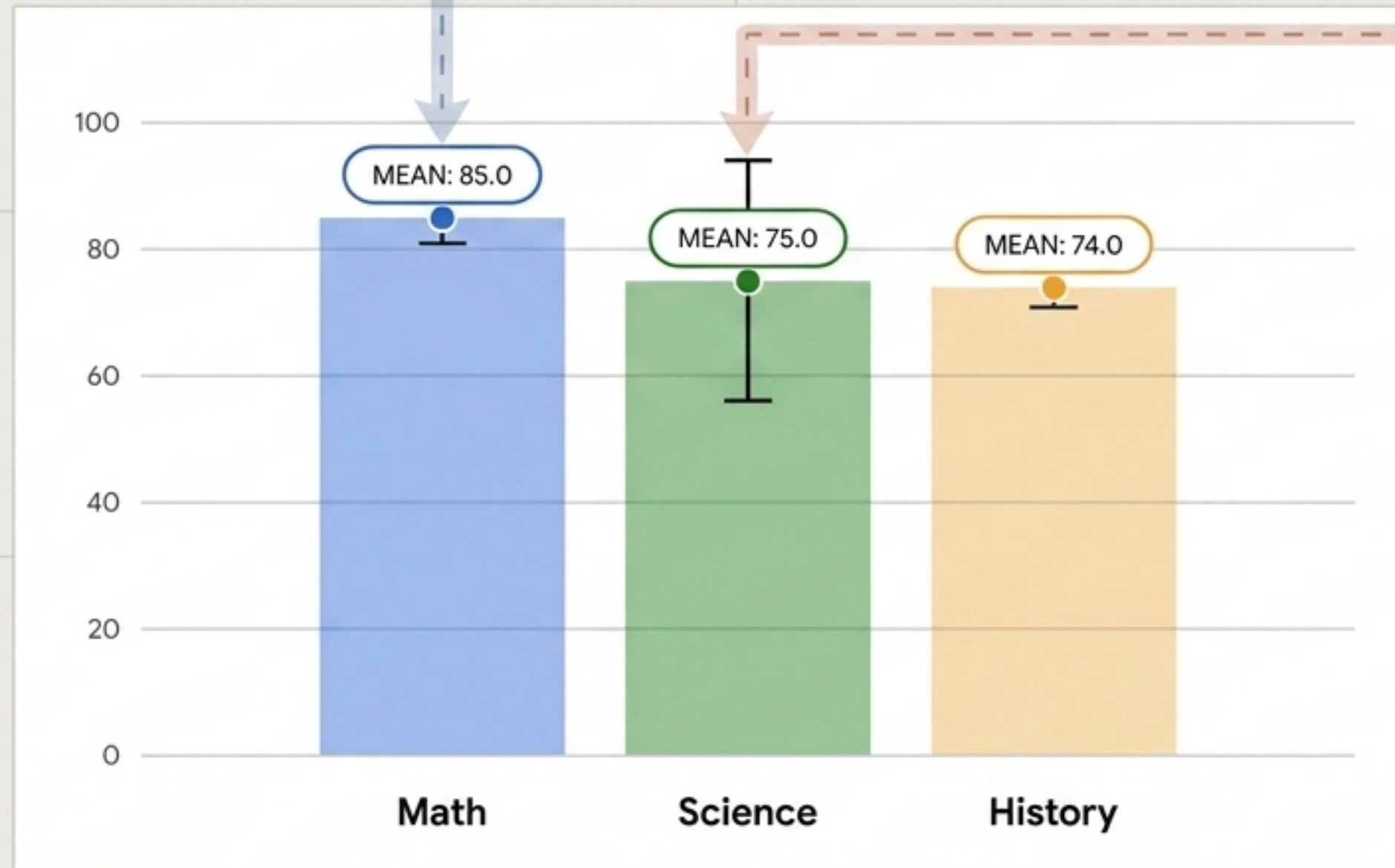
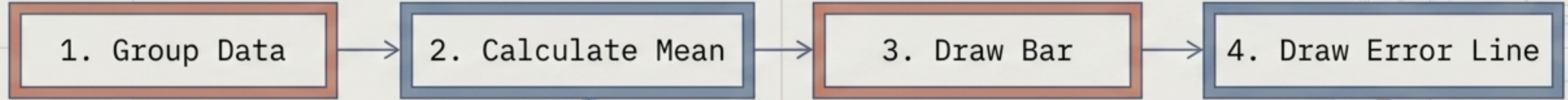


Statistical Calculation (Seaborn)



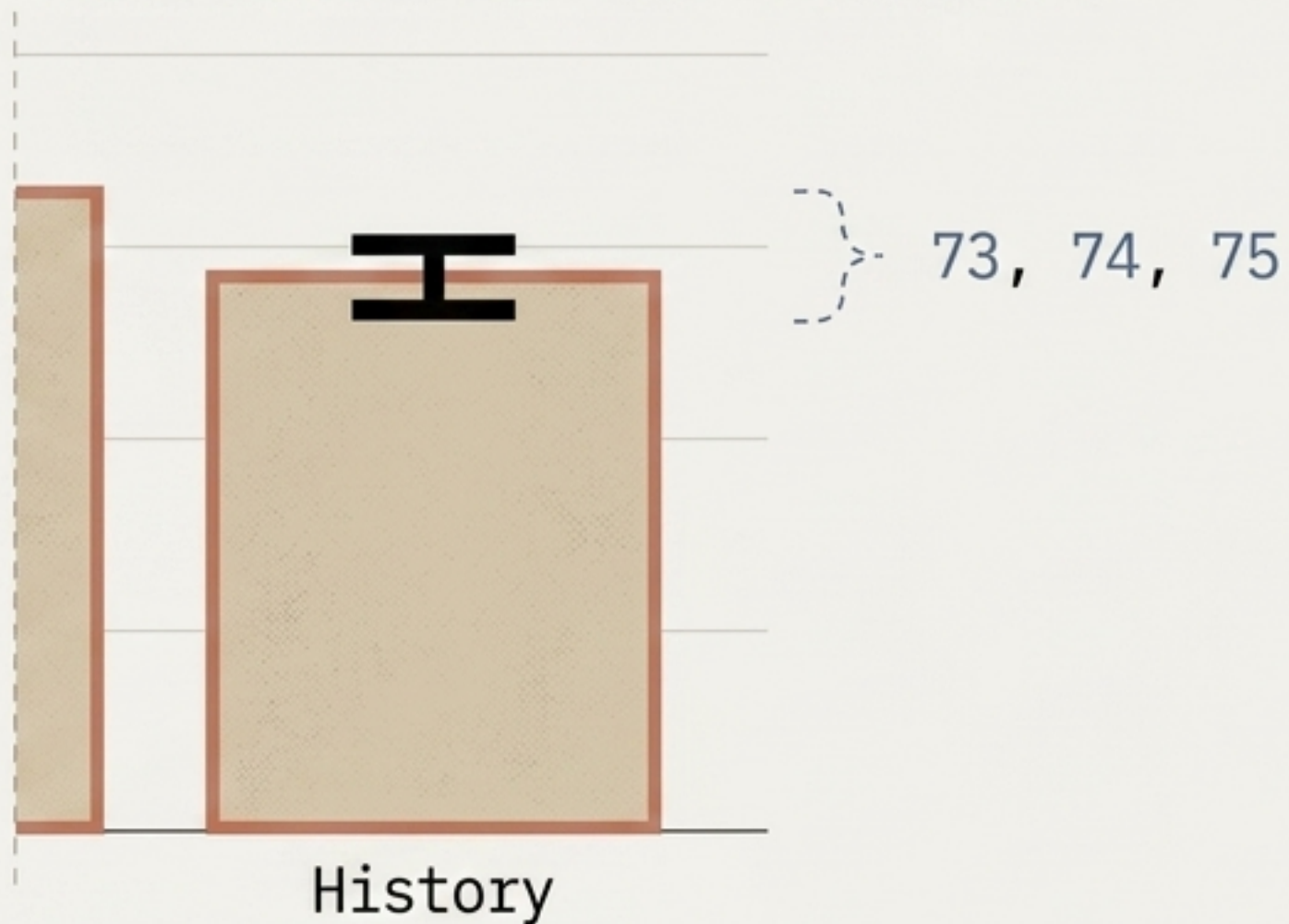
Key Rule: Unlike `countplot()`, `barplot()` absolutely requires BOTH an X and Y axis to function.

Under the Hood: The 4-Step Mechanism



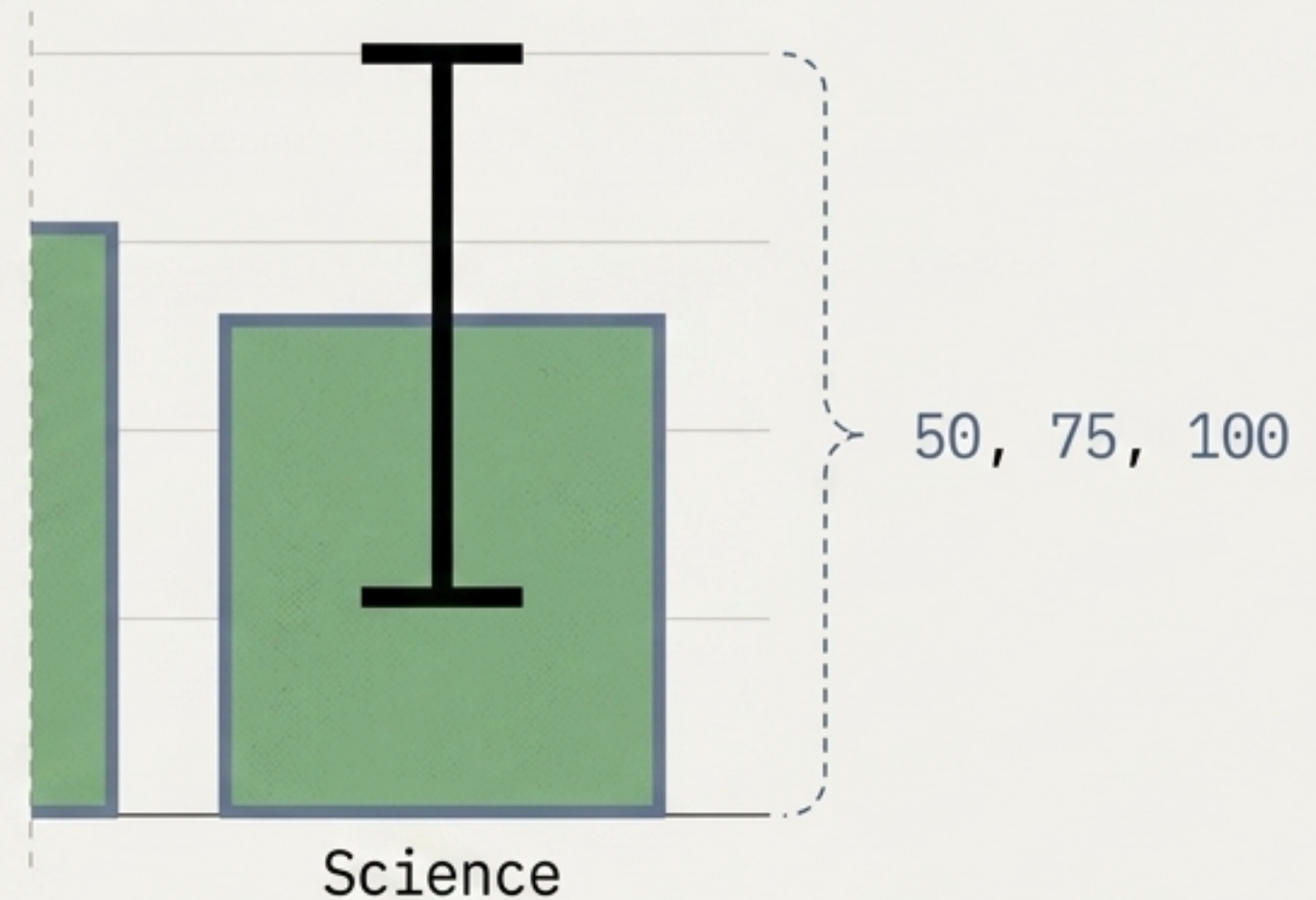
The Mystery of the Error Bar

Left Panel: Editor - Low Variance Diagnostic



Low Variance: Reliable Average.
Everyone got roughly the same score.

Right Panel: Output Console - High Variance Diagnostic

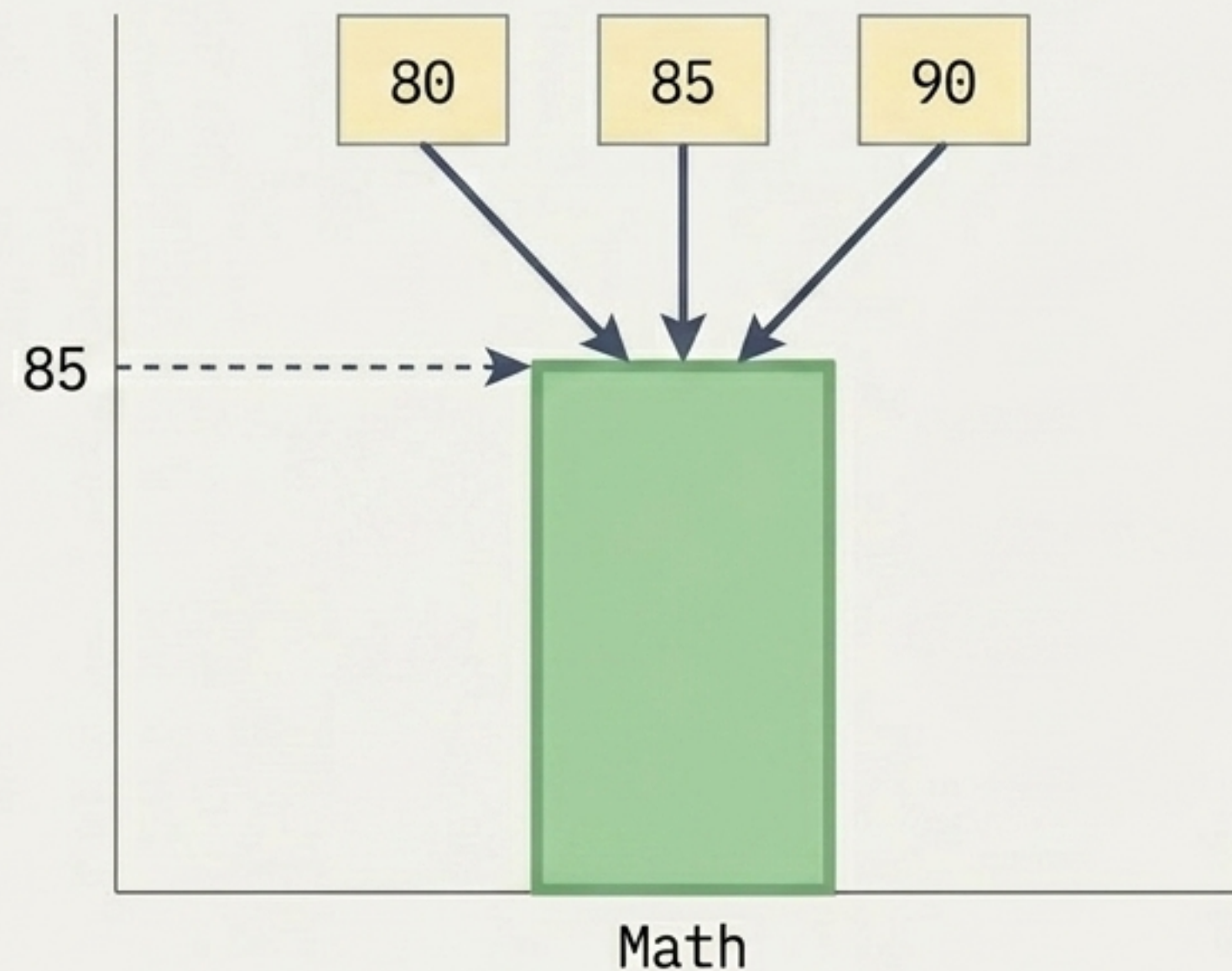


High Variance: Unreliable Average.
Scores were all over the place.

Left Panel: Editor - Code Source

```
data = {  
    'Subject': ['Math', 'Math', 'Math'],  
    'Test_Score': [80, 85, 90]  
}  
  
df = pd.DataFrame(data)  
  
sns.barplot(data=df, x='Subject',  
            y='Test_Score')
```

Right Panel: Output Console - Mean Calculation

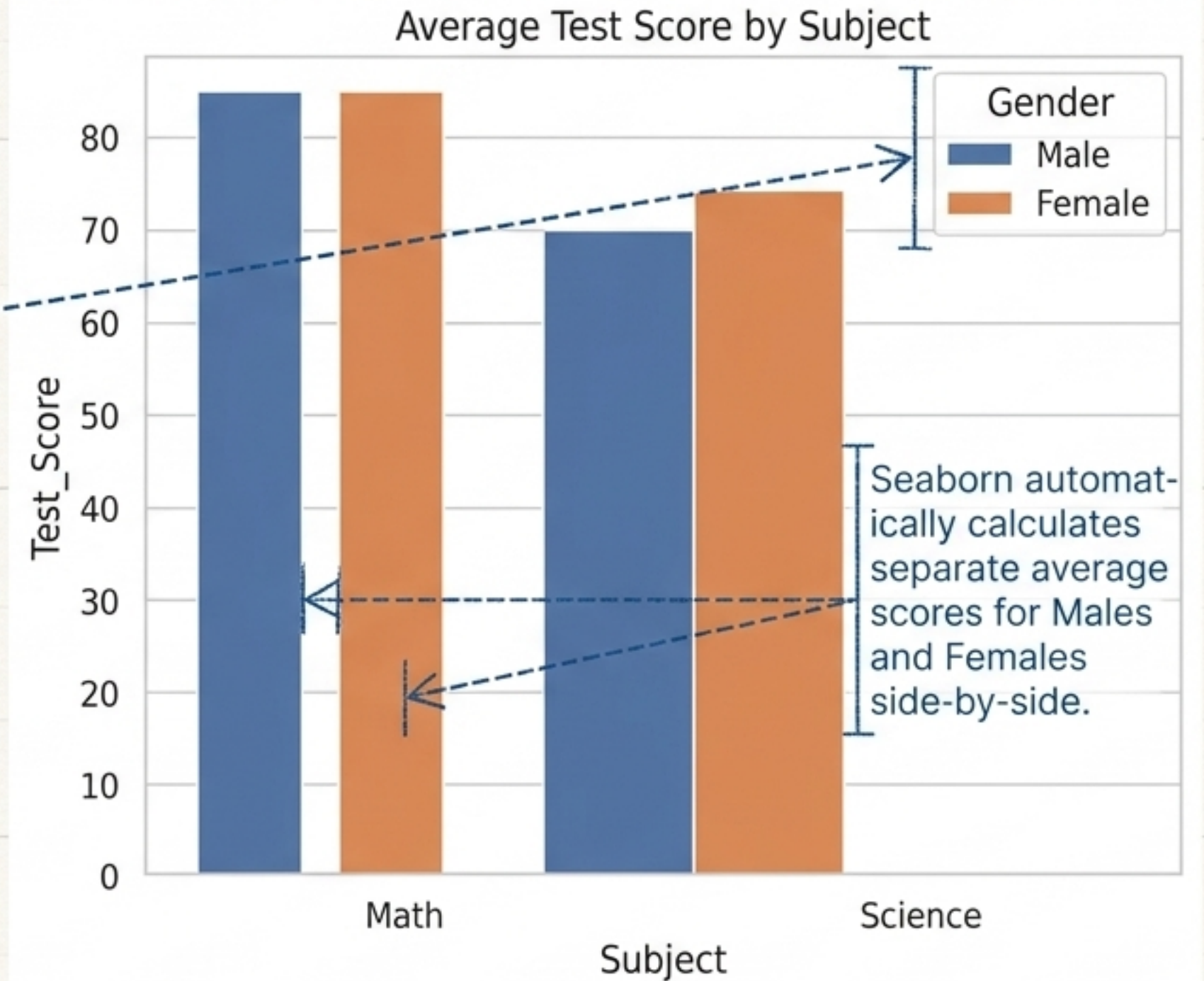


Multiple explicit inputs collapse into a single mathematical mean.

Left Panel: Editor - Code Source

```
sns.barplot(data=df,  
            x='Subject',  
            y='Test_Score',  
            + hue='Gender', errorbar=None  
            )
```

Right Panel: Output Console - Grouped Mean Calculation

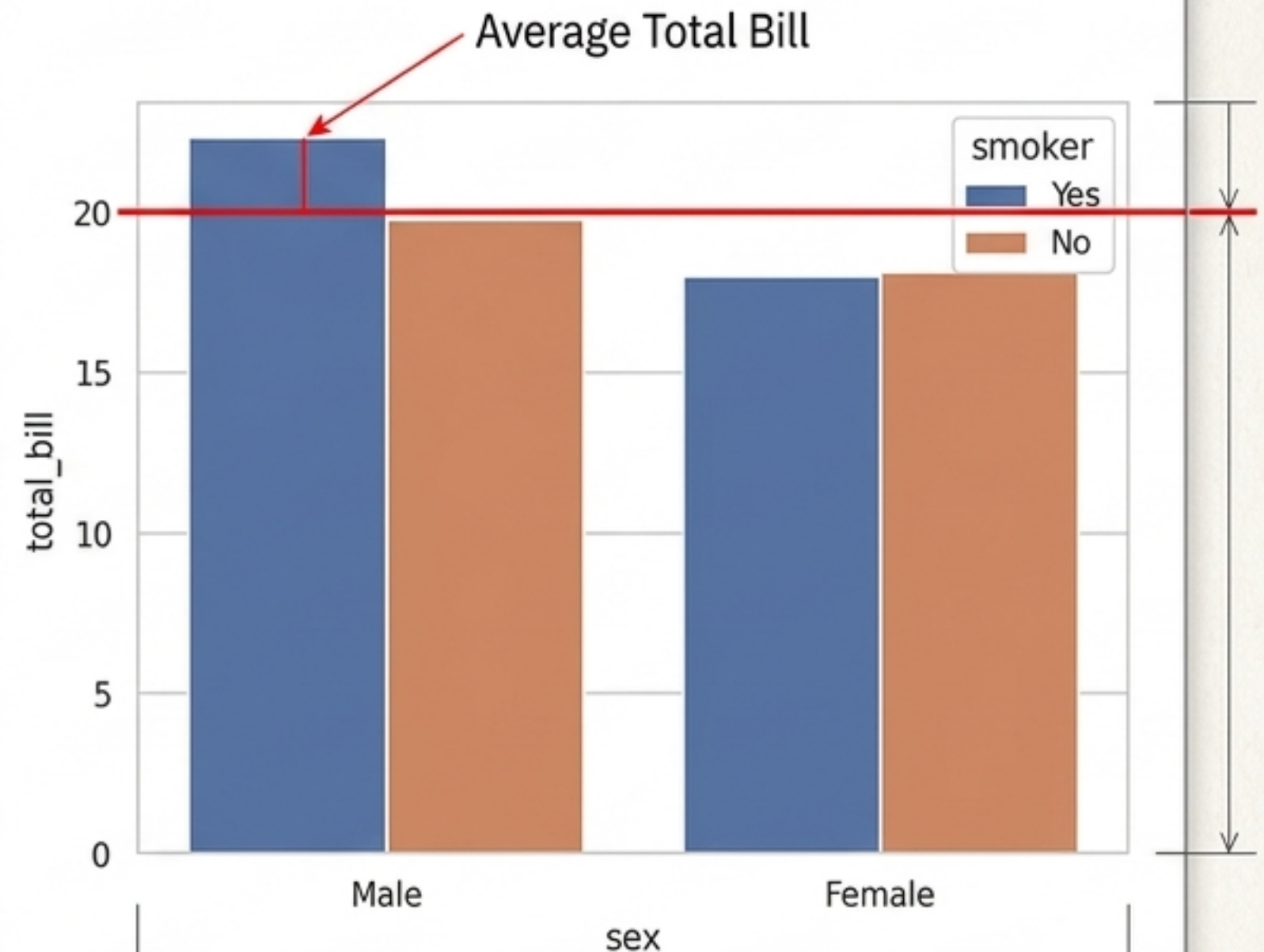


Applied Bar Plot: The Tips Dataset

Left Panel: Editor - Code Source

```
# By default it works with mean value of y data  
sns.barplot(data=tips_data, x='sex',  
            y='total_bill', hue='smoker', errorbar=None)
```

Right Panel: Output Console - Grouped Mean Calculation



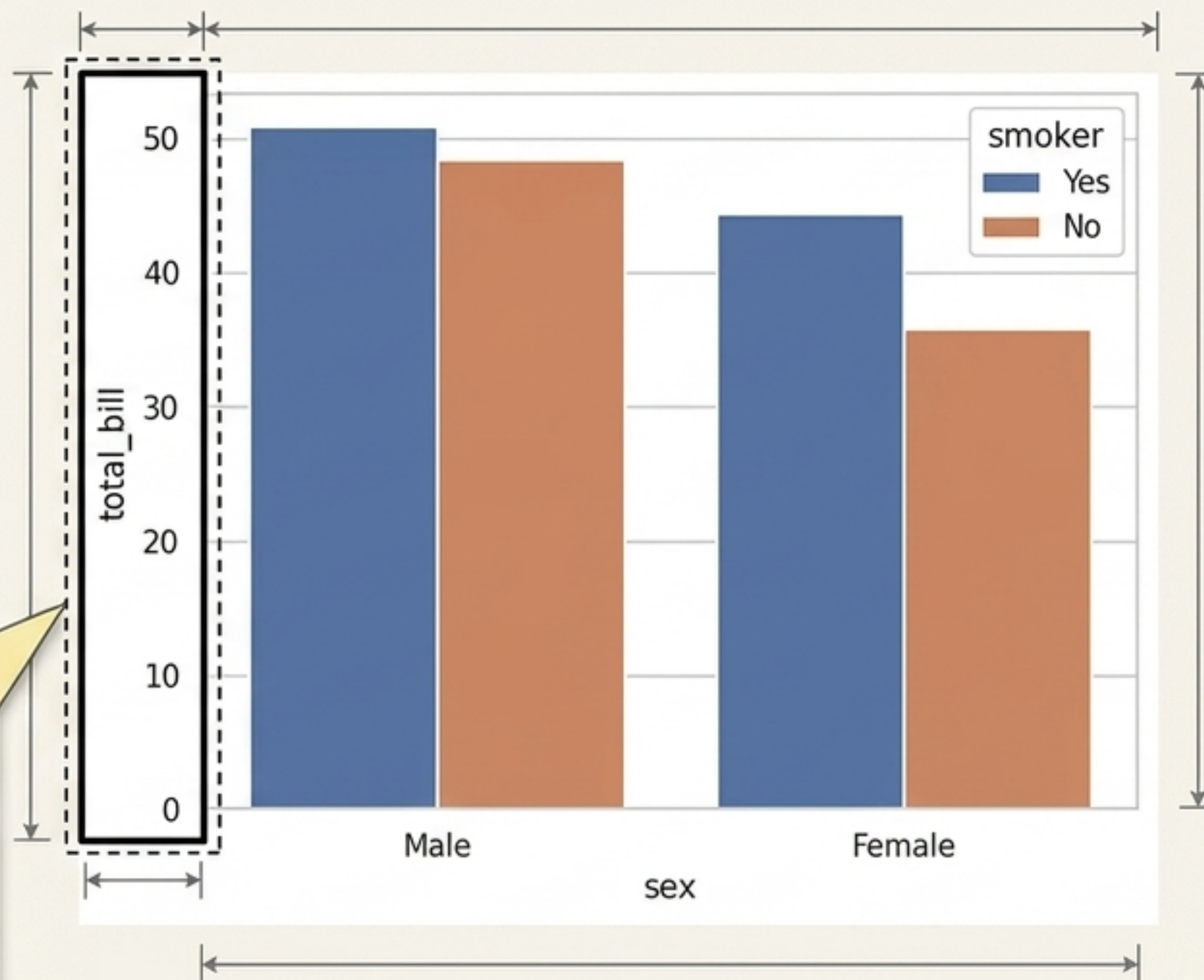
Advanced Control: Changing the Estimator

The Editor

```
# Changing Default to max  
sns.barplot(data=tips_data, x='sex',  
            y='total_bill',  
            hue='smoker',  
            errorbar=None,  
            estimator=max  
            )
```

Notice the scale shift! The Y-axis no longer peaks at 20 (Mean). It now peaks over 50. Seaborn searched the dataset for the single highest bill per group and plotted that exact mx value.

The Output Console



When you need to **Count** (Frequency)

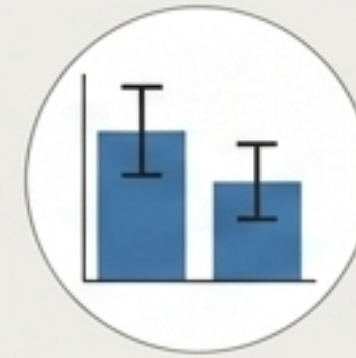
```
sns.countplot(x='Category')
```



Requires 1 Axis.
Calculates Volume.

When you need to **Average** (Stats)

```
sns.barplot(x='Category', y='Value')
```



Requires 2 Axes.
Calculates Math (Means, Max, etc).